Catching card fraud with 1,000 investigations a month

Part A: a fraud detection model. Part B: how it is run, monitored and kept honest.

Xavier Seah | Risk Management division

A behaviour-based model catches 99% of fraud within the monthly budget, and the monitoring loop caught the one real shift

99%

of fraud caught by the top 1,000 alerts a month, on six unseen months

0.98

PR-AUC on the test period (XGBoost, 30 behavioural features)

1

genuine drift alarm in six months, raised in December and explained

The autoencoder was tested as a second model. It did not improve the ranking and is kept only as a drift signal. The recommendation is to deploy XGBoost as a nightly batch job, queue the top 950 scores plus 50 random audits each month, and run the monitoring job on the investigators' verdicts.

Part A
The model

Data, features, validation, results and explainability

The budget covers 1.4% of monthly transactions, so the job is ranking, not classification

1.30M training transactions, Jan 2019 to Jun 2020, 0.58% fraud

556K test transactions, Jun to Dec 2020, held out entirely, 0.39% fraud

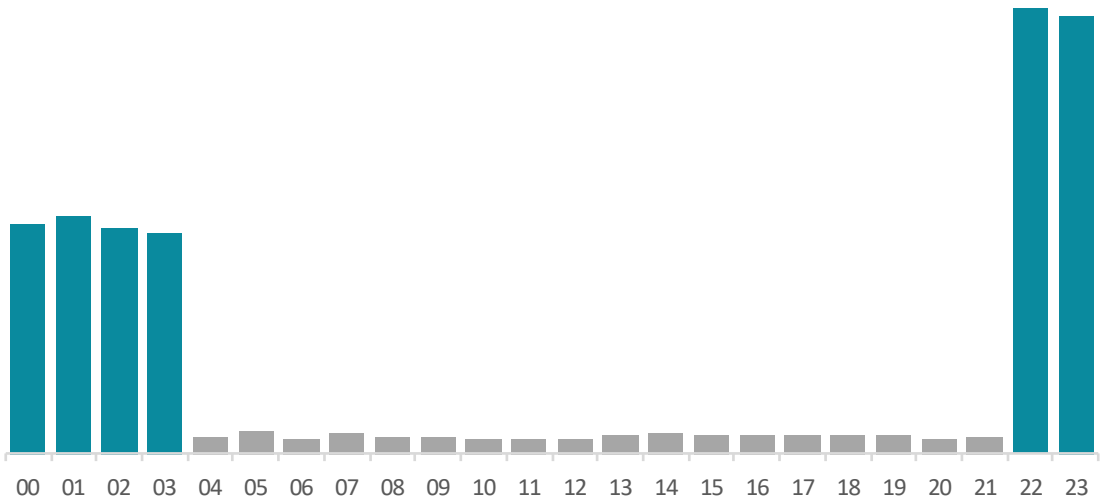
72K transactions a month on average, so 1,000 investigations is the top 1.4%

~300 frauds a month, fewer than the 1,000 we can review, so precision tops out near 30% even when every fraud is caught; recall is the metric to judge by

Implication: the headline metric is PR-AUC (how good is the ranking), and the operating metric is how much fraud the top 1,000 catch each month. Overall accuracy is meaningless at 0.4% fraud.

Fraud is a night-time, high-value event: 22:00 to 03:00 runs at 15 to 25 times the daytime rate

Share of transactions that are fraud, by hour of day (training period)



Daytime: about 0.1%
Late evening (22:00, 23:00): 2.9%
Early hours (00:00 to 03:00): 1.5%

Shopping online, grocery in store and miscellaneous online purchases carry the highest rates by category, and the 250 to 1,000 dollar band holds most of the fraud by amount.

What this says: the signal is in behaviour (when, how much relative to normal, how often), not in who the customer is.

30 features describe behaviour relative to the card's own history, using only the past

Time

hour, day of week, night flag

Fraud clusters after 22:00

Amount vs the card

amount, log amount, ratio to the card's past mean, ratio to its past mean in this category

A 300 dollar grocery shop is normal for some cards and alarming for others

Velocity

transactions in the past 1h, 24h, 7d; amount in the past 24h; seconds since the previous transaction

Bursts of activity on one card

Context

age, gender, log city population, customer-to-merchant distance, 14 category flags

Who and where, without identifying the person

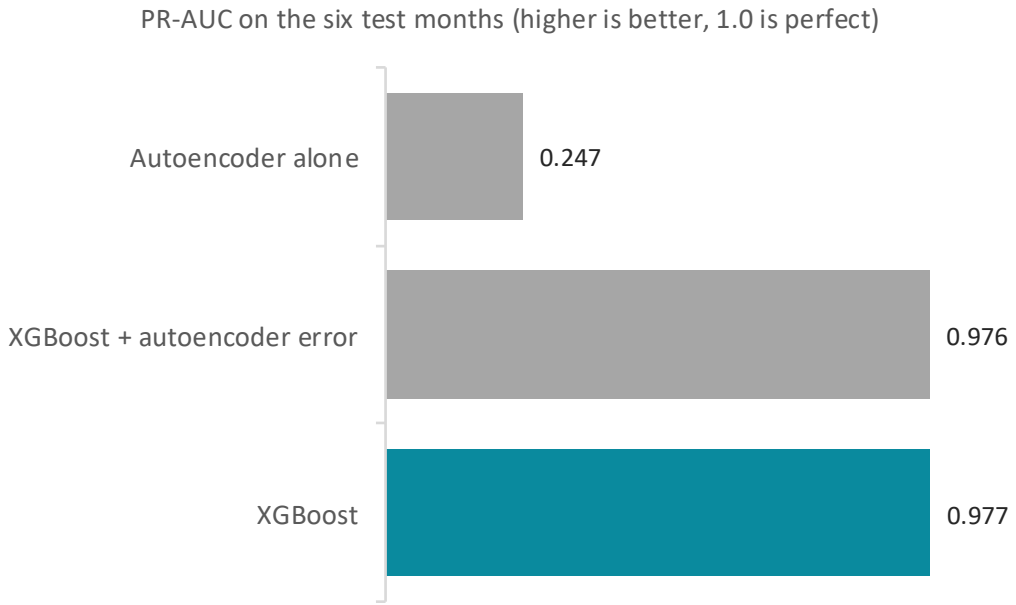
Rules: card numbers, names and transaction ids are never features. History features look backwards only, with train and test stacked so the first test rows see the card's past. A lifetime transaction count was tried and dropped: it only ever grows, so it drifts by construction.

Validation follows time: tune on April to June 2020, then score six months the model has never seen

Fit	Validate	Test
Jan 2019 to Mar 2020 1.10M rows Train XGBoost and the autoencoder	**Apr to Jun 2020** 199K rows Early stopping, alert threshold, precision floor	**Jun to Dec 2020** 556K rows Scored once, replayed month by month in Part B

Why not a random split: card history features would leak the future into training, and a random split hides exactly the thing the business needs to know, which is whether the model still works next quarter. The chronological test also gives Part B a realistic six months to replay.

XGBoost does the work; adding the autoencoder's error as a feature changes nothing

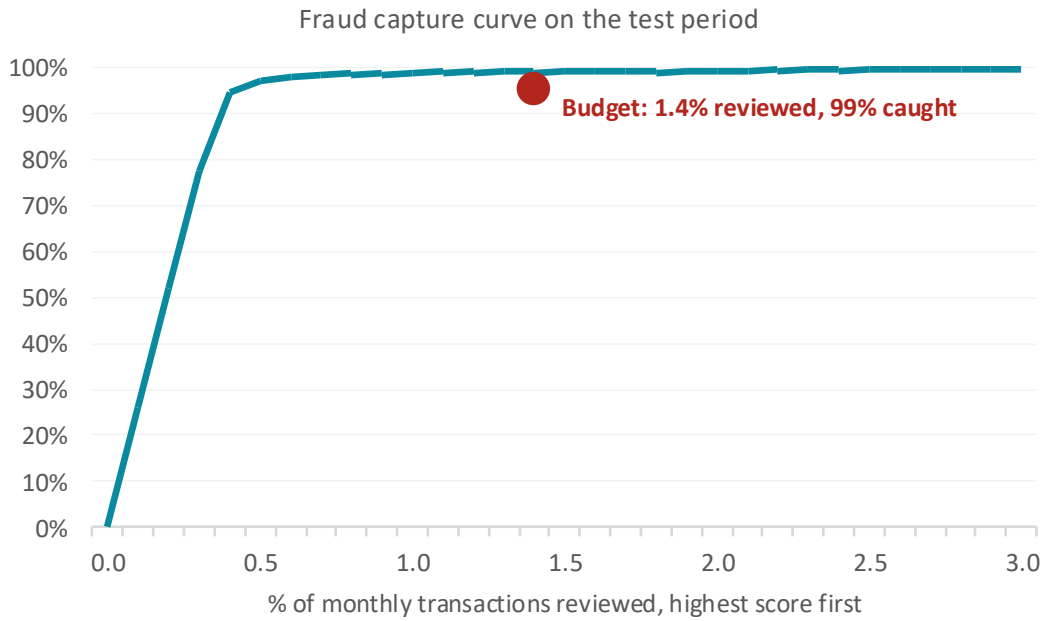


Recall within the 1,000 budget
XGBoost 99.3% | Autoencoder 66.6%

The autoencoder learned "normal" from legitimate rows only. Fraud reconstructs 12 times worse than normal, so it separates the classes, but not sharply enough at the top of the ranking where the budget sits.

Stacking its error into XGBoost moved PR-AUC by 0.001 and flipped sign between validation and test: noise. It stays in the system as a drift signal, not as a model.

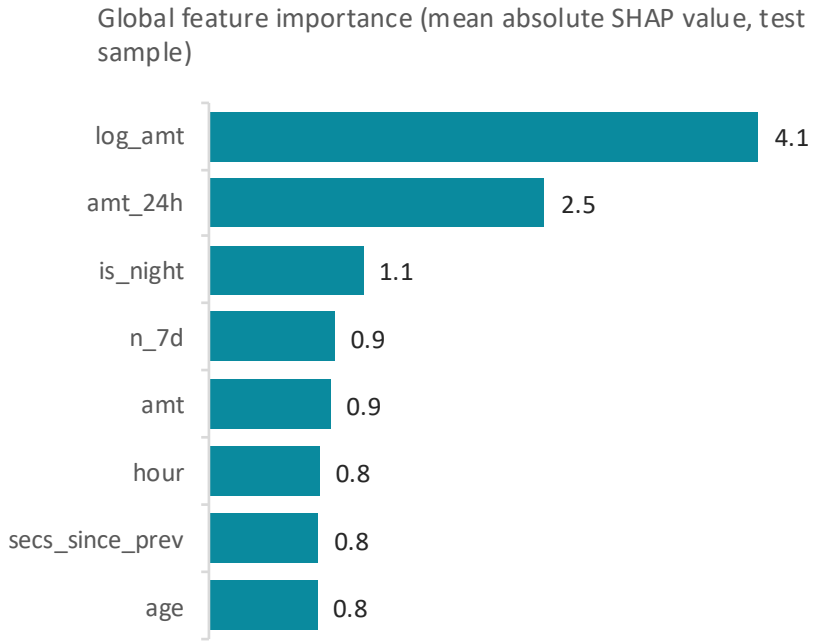
Reviewing 1,000 transactions a month, the top 1.4%, catches 99% of the fraud



	1,000 a month	F1-optimal
Alerts a month	1,000	314
Precision	35%	98%
Recall	99.0%	90.6%
Frauds missed, 6 months	21	201

Two different products. The F1 threshold is what a textbook picks: 314 very confident alerts a month at 98% precision, and 201 frauds let through. The brief says 1,000 investigations are available, so the operating rule is a fixed queue of 1,000 (950 top scores plus 50 random audits), which misses 21 over six months. Figures are from the month-by-month replay in Part B.

Every alert comes with its reasons: amount, 24-hour spend and time of night drive the model



Example alert as the investigator sees it

2020-06-28 23:48 shopping_net $1197.26

Score 1.000. 4 other transactions and $2858 spent on this card in the past 24 hours, at 23:00.

Top reasons (SHAP contribution)

- amt_24h +5.66
- log_amt +5.43
- hour +1.24
- is_night +1.15

Confirmed fraud. Reasons are generated per alert with TreeExplainer, so investigators see why a case was queued, and the global ranking of features goes into the model documentation.

Part B
Operating the model

Scoring, the investigation queue, ground truth, monitoring and drift, retraining

A monthly loop: score, queue 1,000 cases, collect verdicts, monitor, retrain when the monitor says so



Score nightly

Batch job builds features from the day's transactions plus 7 days of card history and loads the champion model from the MLflow registry

Queue monthly

950 highest scores above the budget threshold plus 50 random transactions from below it

Investigate

Verdicts go to a ground-truth table: transaction, selection reason, outcome, date

Monitor

monitor.py reads scores and verdicts: PSI, KS, JS, precision, random-audit rate; logs to MLflow; exits non-zero on alarm

Retrain

Quarterly or on alarm. Challenger registered, promoted by moving the champion alias if it wins on the same validation window

Batch, not an API: the queue is monthly and the features need history. Blocking transactions in real time would be a different product with different latency and feature-freshness demands. MLflow is the system of record: what was trained, what is deployed, what the monitor measured each month.

50 of the 1,000 cases are chosen at random, because labels from alerts alone only tell you what the model already finds

950 top-scored alerts

Measures precision: of what we flag, how much is fraud. On the test months, 27% to 43%.

Cannot measure recall. Every label comes from the model's own view of fraud, so retraining on these alone teaches the model to find what it already finds.

50 random transactions below the threshold

The only unbiased estimate of fraud the model is missing, and therefore the only honest recall figure in production.

On the test months: 0 hits in 50, every month. Consistent with 99% recall at a 0.4% base rate, but noisy. 50 is the minimum worth running; more if the team can spare them.

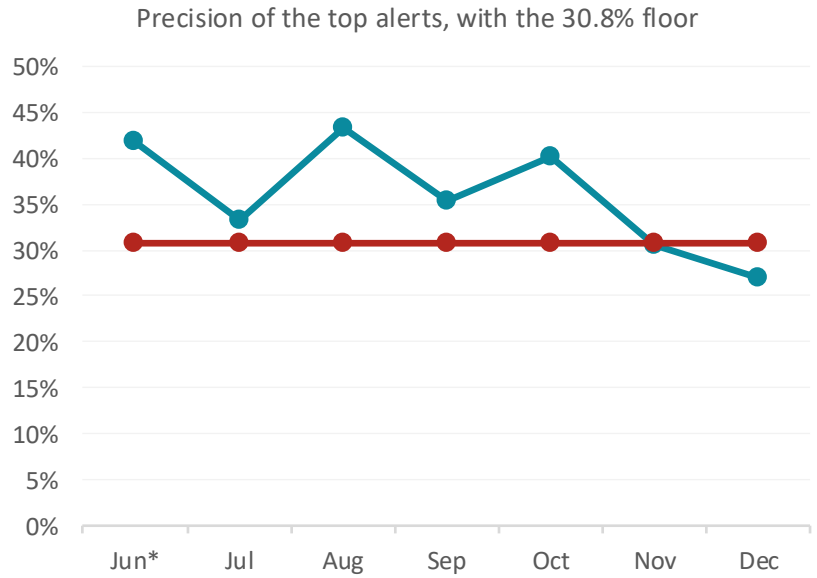
Both groups feed retraining. Recall is always reported from the random audit, never from alert outcomes.

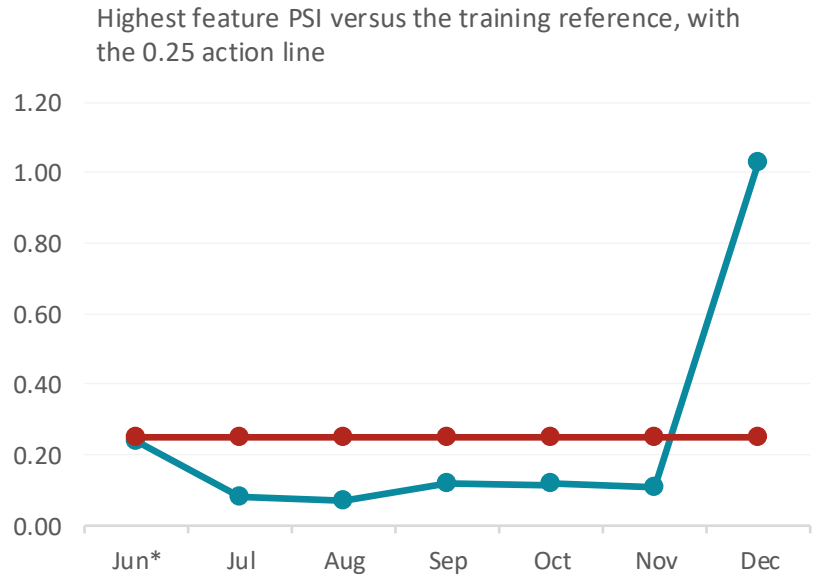
Three questions, three signals: did the inputs move, is the model reacting differently, is the output worse

Question	Signal	Compared against	Alarm
Data drift: did the inputs move?	PSI per feature and on the score; KS statistic on continuous features; JS divergence on category mix	Decile edges, proportions and a 20K-row sample saved at training time	PSI 0.25 (0.10 watch); KS D 0.15 (0.05 marginal); JS 0.10
Model drift: is the model leaning on different features?	Change in mean absolute SHAP per feature, top 5 features	Validation period	Any top-5 feature moves by more than 30%
Performance: is the output worse?	Precision of the top alerts; fraud rate in the random audit	Precision floor = 80% of validation precision at budget (0.308)	Precision below the floor; audit rate rising for 2 to 3 months

Each signal is a number with a threshold, logged to the fraud-monitoring experiment in MLflow every month, so the metrics page is the dashboard. The autoencoder error adds a multivariate check: the share of transactions above the training 99th percentile should stay near 1%.

Replaying six months: the monitor stayed quiet until December, then fired on both data drift and precision





* June is a 10-day partial month with a prorated budget. November scraped under the floor by 0.002, which is noise and a reason to set the floor from several production months rather than one validation window. December is a real alarm: PSI 1.03 on 7-day velocity, precision 27%.

December was seasonality, not decay: volume doubled and velocity features shifted, but the purchase mix did not

140K

transactions in December against 72K in a normal month

11% → 39%

share of transactions with 10 or more on the same card in the past 7 days; this one bin contributes 0.36 of the 1.03 PSI

0.42

KS statistic on 7-day velocity (alarm at 0.15); 0.19 on 24-hour spend and time since previous transaction

0.000

JS divergence on the category mix: the kinds of purchases were unchanged

What the alarm means: the model is being asked about a world it was not trained on, and precision fell to 27%. What it does not mean: the model is broken. The monitor cannot tell seasonality from decay with one year of history; a person can, in a few minutes, from the breakdown above.

Action: compare December with the previous December once one exists, or exclude the holiday weeks from the PSI reference. Either way the alarm did its job.

Retraining is a governed decision: challenger beats champion on the same window, or nothing changes

When	Quarterly on schedule, or earlier when a month raises a data-drift or model-drift alarm, or precision is under the floor two months running
How	Retrain on all labelled data including the random audits; validate on the two most recent labelled months; register as challenger in MLflow
Promote	Only if PR-AUC and precision at budget beat the champion on that window, and the SHAP ranking has been reviewed. Promotion is moving one alias; rollback is moving it back
Record	Every training run, every monthly monitoring run and every promotion lives in MLflow, so any alert can be traced to the model version that raised it

Next steps

Package the scoring job in Docker and schedule it nightly | LightGBM and CatBoost as challengers | Weight evaluation by amount at risk, not counts | Seasonal PSI reference after the first full year

Deploy XGBoost as a nightly batch, spend the 1,000 investigations on the top 950 plus 50 random, and let the monthly monitor decide when to retrain

99% of fraud caught within budget | every alert explained | one real drift alarm in six months, diagnosed as seasonality

Appendix: model configuration and monthly replay

Item	Value
Model	XGBoost, hist tree method, 976 trees at early stopping (max 2000), depth 6, learning rate 0.05, subsample 0.8, scale_pos_weight 172
Features	30: 16 behavioural plus 14 category flags; no identifiers; history features past-only
Autoencoder	30-64-24-8-24-64-30, MSE, Adam 1e-3, 20 epochs on 1.09M legitimate rows; fraud error 12x normal
Thresholds	Budget 0.0043 (about 1,000 alerts a month); F1-optimal 0.93; precision floor 0.308
Tooling	Python, pandas, XGBoost, PyTorch, SHAP, scipy, MLflow (SQLite backend, model registry with champion alias)

Month	Volume	Fraud	Reviewed	Precision	Recall
2020-06	30,058	133	333	41.8%	99.2%
2020-07	85,848	321	1000	33.3%	98.4%
2020-08	88,759	415	1000	43.3%	99.0%
2020-09	69,533	340	1000	35.4%	98.8%
2020-10	69,348	384	1000	40.2%	99.5%
2020-11	72,635	294	1000	30.6%	99.0%
2020-12	139,538	258	1000	26.9%	99.2%

Recall per month is known here only because the test file is labelled; in production it is estimated from the random audit. Code: fraud_detection.ipynb (v1.3) and monitor.py, run from the folder containing fraudTrain.csv and fraudTest.csv.